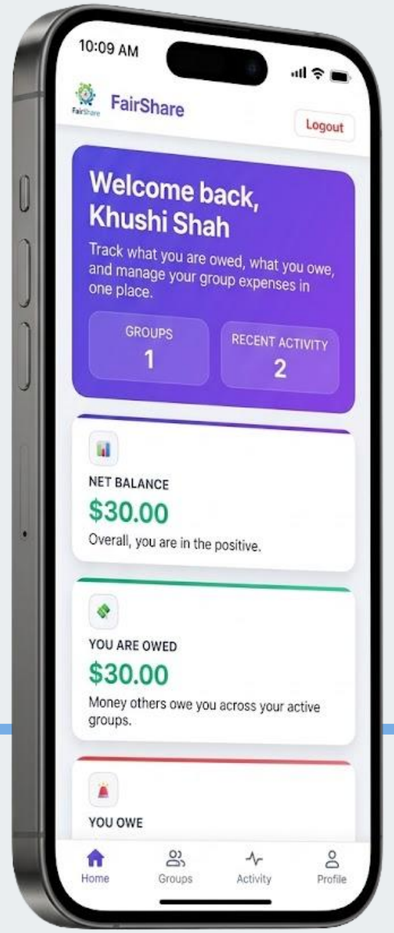
FAIRSHARE SPRINT - 3

KHUSHI SHAH



SPRINT 1: STORIES

01 USER SIGNUP & LOGIN

02 CREATE GROUP

03 JOIN GROUP VIA INVITE CODE

04 ADD EXPENSE

05 EDIT/DELETE EXPENSE

06 EQUAL EXPENSE SPLIT

07 BASIC BALANCE DASHBOARD

TOTAL STORY POINTS: 37

SPRINT 2: STORIES

01 SELECT PARTICIPANTS FOR AN EXPENSE

02 PERCENTAGE-BASED EXPENSE SPLIT

03 ATTACH RECEIPTS TO EXPENSES

TOTAL STORY POINTS: 21

SPRINT 3: STORIES

EXPORT BALANCE SHEET (SP: 5)

- Export balance sheet as PDF/CSV
- Includes all:
  - Users
  - Expenses
  - Net balances
- Data matches dashboard

GENERATE SETTLEMENT PLAN (SP: 8)

- System generates instructions:
  - “A pays B $X”
- Minimizes number of transactions
- Updates balances after settlement

CLEAR ALL DEBTS (SP: 8)

- Option to mark all debts as settled
- Resets balances to zero
- Dashboard reflects cleared state

ZELLE INTEGRATION (SP: 5)

- Add Zelle email / phone
- Upload QR code for payments

SPRINT ESTIMATION & PROJECT COMPLETION

37	21	26	26
SPRINT 1 VELOCITY	SPRINT 2 VELOCITY	SPRINT 3 VELOCITY	MEDIAN VELOCITY

All planned features successfully completed within 3 sprints, achieving MVP goals with 90% confidence interval reliability.

Total sprints required were 3.

No additional sprint is needed

TESTING AND COVERAGE TOOLS (SPRINT 3)

- **JUNIT 5**

Unit testing framework for backend logic verification

- **JACOCO**

Comprehensive code coverage measurement and reporting

- **MOCKITO**

Mocking dependencies including services and repositories

- **DESIGNITEJAVA**

Detect architecture, design and implementation smells

- **GITHUB ACTIONS**

Implemented CI/CD with GitHub Actions to automate testing and ensure consistent code coverage on every push

- **MAVEN SUREFIRE**

Automated test execution across all test suites

- **VITEST**

Frontend logic testing

FairShare Backend

Last Published: 2026-03-16 | Version: 1.0.0 FairShare Backend

Surefire Report

Summary

[Summary] [Package List] [Test Cases]

Tests	Errors	Failures	Skipped	Success Rate	Time
98	0	0	0	100%	4.835 s

Note: failures are anticipated and checked for with assertions while errors are unanticipated.

- Backend tests were executed using Maven Surefire Plugin.
- A total of 98 automated unit tests were run.
- 0 errors, 0 failures, and 0 skipped tests, resulting in a 100% success rate.
- The total test execution time was approximately 4.8 seconds.
- This confirms that the backend functionalities passed all implemented test cases successfully.

FairShare Backend Sessions

FairShare Backend

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.fairshare.service		52%		45%	117	173	197	475	36	72	0	4
com.fairshare.controller		73%		60%	35	91	87	297	0	35	0	3
com.fairshare.model		79%		n/a	14	81	22	104	14	81	0	5
com.fairshare.config		0%		n/a	2	2	8	8	2	2	1	1
com.fairshare.dto		0%		n/a	3	3	3	3	3	3	3	3
com.fairshare		0%		n/a	2	2	3	3	2	2	1	1
com.fairshare.payload		94%		n/a	2	35	2	38	2	35	0	3
Total	1,651 of 4,248	61%	155 of 314	50%	175	387	322	928	59	230	5	20

Created with JaCoCo 0.8.11 202310140853

- Code coverage was measured using the JaCoCo coverage tool integrated with Maven.
- The backend achieved 61% overall code coverage.
- Controller layer coverage reached 73%, indicating strong API testing.
- Service layer coverage is 52%, reflecting testing of core business logic.
- Model and payload classes achieved 79% and 94% coverage respectively.
- Some configuration and DTO classes remain untested because they mainly contain simple data structures or configuration code.

```
wrapping up ...
--Analysis summary--
        Total LOC analyzed: 1563        Number of packages: 8
        Number of classes: 25   Number of methods: 203
-Total architecture smell instances detected-
        Cyclic dependency: 0    God component: 0
        Ambiguous interface: 0  Feature concentration: 0
        Unstable dependency: 0  Scattered functionality: 3
        Dense structure: 0
-Total design smell instances detected-
        Imperative abstraction: 0       Multifaceted abstraction: 0
        Unnecessary abstraction: 3      Unutilized abstraction: 9
        Feature envy: 0 Deficient encapsulation: 0
        Unexploited encapsulation: 0    Broken modularization: 1
        Cyclically-dependent modularization: 0  Hub-like modularization: 0
        Insufficient modularization: 3  Broken hierarchy: 0
        Cyclic hierarchy: 0     Deep hierarchy: 0
        Missing hierarchy: 0    Multipath hierarchy: 0
        Rebellious hierarchy: 0 Wide hierarchy: 0
-Total testability smell instances detected-
        Hard-wired dependency: 3        Global state: 0
        Excessive dependency: 0 Law of Demeter violation: 5
-Total implementation smell instances detected-
        Abstract function call from constructor: 0      Complex conditional: 0
        Complex method: 4       Empty catch clause: 0
        Long identifier: 0      Long method: 0
        Long parameter list: 1  Long statement: 42
        Magic number: 14        Missing default: 0
-Total test smell instances detected-
        Assertion roulette: 0   Missing assertion: 0
        Empty test: 0   Ignored test: 0
        Constructor initialization: 0   Eager test: 0
        Exceptional handling: 0 Conditional test logic: 0
```

- Analyzed 1563 LOC, 25 classes, 203 methods using DesigniteJava
- No major architecture smells → system is well-structured
- Minor issues found (long statements, unused abstractions) → scope for improvement

Vitest Dashboard

Sort by Default

Search... (e.g. test name, tag:expression)

Filter

Fail Pass Skip Only Tests

Slow (>300ms)

FAIL (0) / RUNNING (0)
PASS (1) / SKIP (--)

- src/GroupDetails.test.jsx 3ms
  - FairShare Frontend Logic Tests 2ms
    - should show "You" when payer is the curren... 1ms
    - should return green color for positive bala... < 1ms
    - should format currency to two decimal pla... < 1ms
    - should correctly identify ADMIN role < 1ms
    - should validate a 6-character invite code < 1ms

5	0	5
Pass	Fail	Total

Files	1
Pass	1
Time	511ms

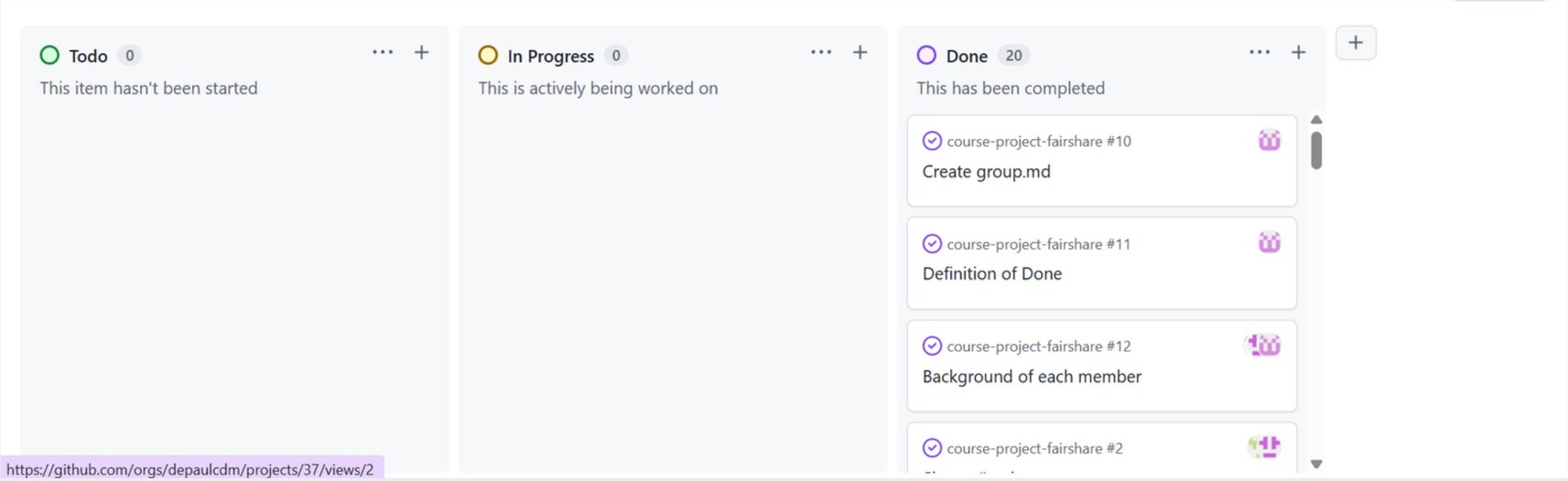
Frontend logic testing was implemented using Vitest. The tests verify UI logic such as user display names, role identification, currency formatting, and invite code validation.

5 workflow runs Event Status Branch Actor

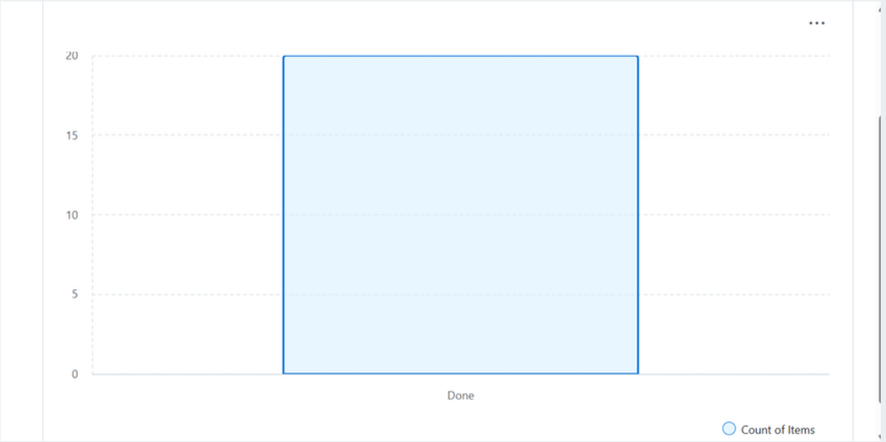
Fix frontend test environment compatibility main Mar 16, 2:55 PM CDT
FairShare CI Build #11: Commit 46ce9f8 pushed by KhushiShah-depaul 58s

Fix FairShareBackendTests syntax error main Mar 16, 2:47 PM CDT
FairShare CI Build #10: Commit e61b670 pushed by KhushiShah-depaul 55s

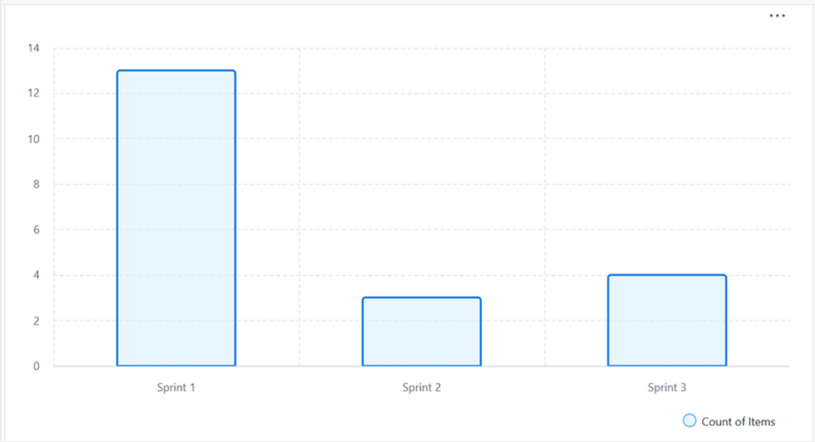
- Github Actions show both successful and unsuccessful builds



PLANNING BORAD IN GITHUB



ALL PLANNED TASKS WERE SUCCESSFULLY COMPLETED, REFLECTING A 100% SPRINT COMPLETION RATE.



SPRINT 1 HAD THE HIGHEST WORKLOAD, FOLLOWED BY A DROP IN SPRINT 2 AND A SLIGHT INCREASE IN SPRINT 3, SHOWING STABILIZED PROGRESS

OUR DEVELOPMENT PROCESS

Agile-Based Development Lifecycle

01 REQUIREMENT GATHERING

02 SPRINT PLANNING

03 SYSTEM DESIGN

04 DEVELOPMENT

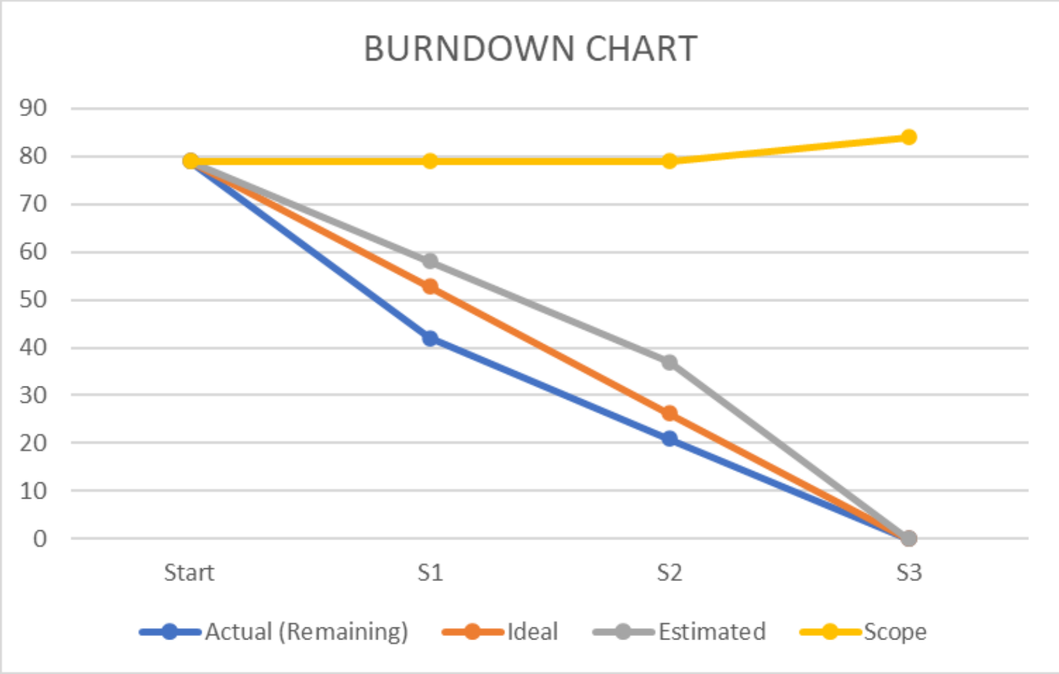
05 TESTING

06 CONTINUOUS INTEGRATION

07 REVIEW & RETROSPECTIVE

The burndown chart shows steady progress across sprints, with a scope increase in Sprint 3 that was successfully completed within the release.

BURNDOWN CHART



KEY METRICS - ALL SPRINTS

SPRINT 1

•Planned Velocity: 37 points
•Actual Velocity: 37 points
•Sprint Predictability Rate: 100%
•Sprint Addition Rate: 0%
•Sprint Completion Rate: 100%
All actions were kept and evaluated

SPRINT 2

•Planned Velocity: 21points
•Actual Velocity: 21 points
•Sprint Predictability Rate: 100%
•Sprint Addition Rate: 0%
•Sprint Completion Rate: 100%
All actions were kept and evaluated

SPRINT 3

•Planned Velocity: 21 points
•Actual Velocity: 26 points
•Sprint Predictability Rate: 81%
•Sprint Addition Rate: 24%
•Sprint Completion Rate: 100%
One additional action was added and evaluated

Overall, Sprint 1 and Sprint 2 demonstrated strong planning accuracy with 100% predictability and completion. In Sprint 3, although predictability decreased due to the addition of a new feature, the team successfully adapted and delivered all planned and additional work. This highlights our ability to balance planning discipline with flexibility, ensuring both consistency and value delivery throughout the project.

SPRINT 3 - RETROSPECTIVE

STOP

- Adding new features mid-sprint without buffer planning
- Slight deviation from initial sprint commitment
- Relying more on manual coordination instead of structured tracking

START

- Early discussion before introducing scope changes
- Improving estimation accuracy for dynamic features
- Strengthening communication between frontend & backend

CONTINUE

- Completing all committed and additional tasks
- Using CI/CD tools (GitHub Actions, testing pipelines)
- Maintaining transparency through regular updates

THANK YOU

DEMONSTRATION: https://youtu.be/GIe5kEu0w4Q